

Beyond Requirements: Software Making as Art

Rob Austin and Lee Devin

Quality is meeting requirements—or is it? Rob Austin and Lee Devin challenge this familiar metaphor for requirements and introduce a new one based on their experience in an industry that seems far from software development—or is it? What are your opinions on this subject? —Suzanne Robertson

No one paying attention to software development recently could miss the raging methodology war. On one side you find *formalists*, who remain fond of the word “process,” emphasize detailed planning to produce quality products, and aspire to a level of repeatability



that evokes manufacturing imagery. On the other side you find *agilists*, who value “individuals and interactions over processes and tools” and “responding to change over following a plan” (“Manifesto for Agile Software Development,” <http://agilemanifesto.org>) and who dislike manufacturing imagery applied to knowledge work.

Each camp likes to describe the other in extreme terms. Agilists highlight the shortcomings of waterfall methodologies, with which they identify formalists. According to agilists, the formalist approach’s orientation toward planning means that by the time you finish a system, the problem it solves has

changed; you might build software with few defects this way, but when you’re done, it’s the wrong software. Formalists accuse agilists of trying to turn back time to when developers started by immediately slinging code. Many systems built this way evolved into hairballs of complex, unmanageable code, formalists say; without plenty of planning up front, you’ll invariably make a mess.

The notion of requirements is central to this debate. Formalists can’t see how agilists expect to build anything that works without thoroughly analyzing requirements. Agilists can’t see the point in spending a lot of time analyzing requirements because they’re going to change anyway.

This conflict is nothing less than growing pains associated with a transition from an industrial to a knowledge economy. Even as they try to move the debate forward, combatants bang repeatedly against a conceptual wall propped up by the word requirements. As we explain this, we’ll inevitably reveal where our sympathies lie, so here they are: We think agilists are at the leading edge of an important shift in how we will manage work in a knowledge economy. It’s hard to see this right now because the metaphor the word “requirements” evokes tilts the battlefield against agilists.

The need for a new metaphor

Formalist software developers can muster to their cause many words we’ve rightly asso-

ciated with the developed world's economic successes since the Industrial Revolution—structure, rigor, and repeatability, for example. The formalist methods' closeness to these reassuring words leaves agilists out in the cold, vulnerable to labeling with a very different set of words—unstructured, loose, and ad hoc. Agilists insist their methods are none of these things, but they lack a solid history. Formalists can point to industrial successes in manufacturing. Agilists have no equivalent success story to invoke. Or so they think. We believe they haven't looked enough.

There is an “industry” that routinely develops new products under harsh deadlines and produces valuable outcomes that operate with great precision even though it's impossible to anticipate those outcomes. This industry's methods are rigorously structured and highly reliable, although they might appear ad hoc or chaotic to uninitiated observers. The industry, which we suggest as a model for software development, is the collaborative arts—more specifically, theater.

This suggestion might appear outrageous at first. However, a good theater, operating under a deadline as firm as any in business (opening night), creates a product that, although uniquely and intentionally innovative in every run, nevertheless finishes within 30 seconds of the same time every performance. The product has a form no one could have predicted. It adapts each night to audience differences and other changing aspects of performance just as an agile development process adapts to changing requirements.

For four years, we've studied collaborative artists in theater and compared their practices to those emerging in agile software development. The similarities in structure, methods, and ways of discussing work are startling. Both theater production and software development are special cases of what we call *artful making*, the emerging model for work in a knowledge economy. Many people assume that economic value creation is becoming more scientific. In places, this is true.

The broader tendency, though, is for work to become more like art.

The structure of artful making

Artful making, whether in software development or theater, has a recognizable structure. It's iterative and thus involves doing and doing again, trying things many different ways. It deemphasizes planning in favor of learning from experience. It delegates control to makers themselves, aspires to enhance individual and idiosyncratic talents, and uses the environmental uncertainty of ongoing change to fuel innovation.

A theater rehearsal makes many prototypes of the ultimate product; each one feeds back into the process—an exponential, unpredictable, creative cycle. The final product appears as a result of the work, not as a preconceived goal the work achieved. In rehearsal, the actors play a scene, discuss it with the director and each other, then play it again. They retain both the first version and the discussion as materials from which to make the next version. That next version develops, moves forward, and creates new features from consultation and iteration.

Agile software developers follow a similar process. They build a rough prototype quickly and take it to the customer for discussion. Everyone

knows this is step one, not the end. It's not a mere unfinished version of a final product, but a step in the product's development. The developer and the customer play with the prototype, during which time the developer gets new ideas about the larger problem, and the customer discovers possibilities that didn't exist until created from this interaction. The resulting product answers questions no one knew how to ask initially.

This cycle (consult, build, consult, build some more) can continue as long as time permits. At some point before a deadline, the prototype will be good enough for the customer. Then, the iterations can end, or they can continue to explore avenues opened up in the process, generated by the act of making. The whole transaction changes from finding to making, from problem-solving to creativity.

Artful making is not always the best approach, even in knowledge work. Where prototypes are too expensive to build or where failed prototypes have great consequences, formalist methods remain the best choice. But in software development and other knowledge work, information technologies increasingly enable simulation, rapid prototyping, and low-cost testing and consequence assessment. The technologies lower the costs of working iteratively and bring us to the brink of a new way of organizing work for the knowledge economy.

Why “requirements” is too limiting

Many people not involved in theater suppose that playmaking intends to reproduce the author's intentions precisely. If this were true, the script would be a specification and the actors would be like assembly line workers trying to build a product to conform to the spec. But it is not remotely true. A script is not detailed enough to be a specification; it contains words to be spoken and a few stage directions, but the actors, designers, and the director must make the majority of choices. The words in the script are like high-level constraints for a software system, the

Formalists can point to industrial successes in manufacturing. Agilists have no equivalent success story to invoke. Or so they think. We believe they haven't looked enough.

equivalent to an accounting system's *generally accepted accounting principles* or a medical system's government regulations. In those too, most of the choices that make the system good or bad are left to developers, designers, and managers.

The difference between a good and bad system is not how well it meets the requirements you know in advance. Meeting requirements is a necessary but insufficient condition for producing an excellent system. What makes a system great is details that

are not specifiable in advance—aspects that must evolve in the making. A production of Shakespeare's *Macbeth* is not great merely because the actors said all the words perfectly. To be successful, the play must have vitality, a life that is created every night anew. Similarly, a system must do more than meet requirements known in advance. As long as our processes are structured for conformance to advance specs, we'll have problems because software development's essence is creation, not conformance. That doesn't—we repeat, doesn't—mean the process can't be rigorous. It's time to stop defining software develop-

ment's success in terms of meeting requirements. It just doesn't work that way. In a rapidly changing world, it can't. 

Rob Austin is an assistant professor at the Harvard Business School. Contact him at Morgan T91, Soldiers Field, Boston, MA 02163; raustin@hbs.edu.

Lee Devin is a dramaturg at the People's Light and Theater Company in Malvern, Pa. Contact him at People's Light, 39 Conestoga Rd., Malvern, PA 19355; ldevin1@swarthmore.edu.

They coauthored *Artful Making: What Managers Need to Know About How Artists Work* (Prentice-Hall/Financial Times), to be published in the spring.

Career Opportunities

UNIVERSITY OF COLORADO AT COLORADO SPRINGS

The University of Colorado at Colorado Springs seeks a renowned scholar in Computer Science, preferably in the areas of software engineering or computer communications and networks. This position is endowed by El Pomar Foundation, one of the largest private foundations in the west. El Pomar Foundation creates opportunities for Colorado's future through proactive philanthropy and innovative partnerships.

This position offers a unique opportunity for a candidate to have two related leadership roles. Twenty-five percent of the El Pomar Professor's time will be devoted to bringing technical expertise to CITTI, the University affiliated Colorado Institute for Technology Transfer and Implementation. The remaining seventy-five percent of the El Pomar Professor's time will be as a faculty member with the Computer Science Department.

To see a detailed description of the position please go to <http://csweb.uccs.edu/positions/elpomarat.htm>

Candidates must have the qualifications, experience, and professional stature appropriate for the academic rank of professor of computer science and a demonstrated commitment to excellence in research and teaching. Applicants should send a letter describing qualifications and experiences, curriculum vita, and contact information for five references. Salary and other support costs are negotiable.

Applications and nominations should be directed to the El Pomar Pro-

fessor Search Committee, Department of Computer Science, University of Colorado at Colorado Springs, P.O. Box 7150, Colorado Springs, CO 80933. Applications for this position may be sent electronically to a confidential e-mail address, elpomarsearch@cs.uccs.edu.

Applications will be reviewed beginning February 1, 2003 and continue until the position is filled. The University of Colorado is an Affirmative Action/Equal Opportunity employer and encourages a diversity of applicants.



CIGITAL LABS Opportunities in the Research Labs

Cigital makes software better. Our solutions help companies increase productivity while mitigating the business risks inherent in software. At Cigital Labs, we specialize in researching, developing and applying next-generation technologies that help our clients repeatedly develop and deliver software better and faster. We are currently searching for exceptional researchers at all levels to become integral parts of our research team.

Director of Science: Will be responsible for determining the direction of scientific and technical research at the labs. Successful candidate must have at least ten years of experience winning, managing and growing cutting-edge research initiatives. Must possess strong scientific research skills and a proven track record of: (1) winning research contract/grants, (2) publishing peer-reviewed articles, and (3) pushing forward an organization's overall research agenda.

Research Scientist: Successful can-

didate must have a Ph.D. in Computer Science and have performed noteworthy research in the field of software assurance. Excellent writing skills, a good publishing record and demonstrated potential for obtaining external research funding are required. Must be willing to mentor junior staff members.

Research Associate: Successful candidate must have a Masters in Computer Science (or equivalent) with a strong background in core computer science areas. Must possess excellent research and analytical skills and the wherewithal to tackle difficult, open-ended problems. Must be a self-motivated, quick learner with the ability and interest to rapidly absorb new technologies.

Our researchers must be familiar with cutting-edge technologies in the fields of software assurance (reliability, safety, security), software engineering (OOA/D, test planning, execution and automation) or software analysis. In addition to performing research, candidates at all levels are responsible for investigating new areas for proposals, writing and presenting papers, and pushing the technology envelope.

Cigital is located in Northern Virginia, just minutes outside of Washington, DC, in one of the nation's fastest growing technology regions. As the leading authority in the field of software assurance, Cigital offers an excellent salary and benefits package, along with a casual, relaxed atmosphere centering around highly motivated teams.

If you are committed to excellence and want to be involved with tomorrow's technology today, visit <http://www.cigitallabs.com> and email your resume to jobs@cigital.com. Cigital Inc., 21351 Ridgetop Circle, Suite 400, Dulles, VA 20166.

Reproduced with permission of the copyright owner. Further reproduction prohibited without permission.